

Executable and Linkable Format

In computing, the **Executable and Linkable Format**^[3] (**ELF**, formerly named **Extensible Linking Format**) is a common standard file format for executable files, object code, shared libraries, device drivers, and core dumps. First published in the specification for the application binary interface (ABI) of the Unix operating system version named System V Release 4 (SVR4),^[4] and later in the Tool Interface Standard,^[1] it was quickly accepted among different vendors of Unix systems. In 1999, it was chosen as the standard binary file format for Unix and Unix-like systems on x86 processors by the 86open project.

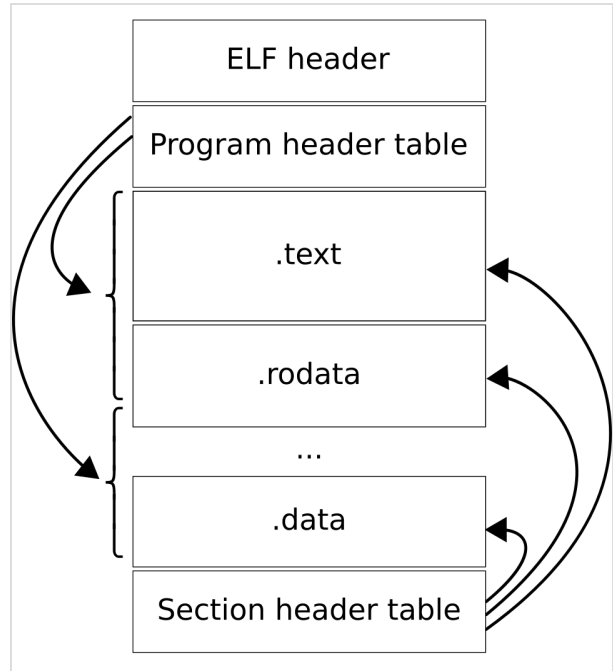
By design, the ELF format is flexible, extensible, and cross-platform. For instance, it supports different endiannesses and address sizes so it does not exclude any particular CPU or instruction set architecture. This has allowed it to be adopted by many different operating systems on many different hardware platforms.

File layout

Each ELF file is made up of one ELF header, followed by file data. The data can include:

- Program header table, describing zero or more memory segments
- Section header table, describing zero or more sections
- Data referred to by entries in the program header table or section header table

Executable and Linkable Format	
Filename extension	none, .axf, .bin, .elf, .o, .out, .prx, .puff, .ko, .mod, .AppImage and .so
Magic number	0x7F 'E' 'L' 'F'
Developed by	Unix System Laboratories ^{[1]:3}
Initial release	14 May 1998
Latest release	4.2 ^[2] 2025
Type of format	Binary, executable, object, shared library, core dump
Container for	Many executable binary formats



An ELF file has two views: the program header shows the *segments* used at run time, whereas the section header lists the set of *sections*.

ELF header^[5]

Offset		Size (bytes)		Field	Purpose
32-bit	64-bit	32-bit	64-bit		
0x00		4		e_ident[EI_MAG0] through e_ident[EI_MAG3]	0x7F followed by ELF(45 4c 46) in ASCII; these four bytes constitute the <u>magic number</u> .
0x04		1		e_ident[EI_CLASS]	This byte is set to either 1 or 2 to signify 32- or 64-bit format, respectively.
0x05		1		e_ident[EI_DATA]	This byte is set to either 1 or 2 to signify little or big endianness, respectively. This affects interpretation of multi-byte fields starting with offset 0x10.
0x06		1		e_ident[EI_VERSION]	Set to 1 for the original and current version of ELF.
0x07		1		e_ident[EI_OSABI]	Identifies the target operating system <u>ABI</u> . Value ABI
0x08		1		e_ident[EI_ABIVERSION]	Further specifies the ABI version. Its interpretation depends on the target ABI. Linux kernel (after at least 2.6) has no definition of it, ^[6] so it is ignored for statically linked executables. In that case, offset and size of EI_PAD are 8. glibc 2.12+ in case e_ident[EI_OSABI] == 3 treats this field as ABI version of the <u>dynamic linker</u> . ^[7] it defines a list of <u>dynamic linker's</u> features, ^[8] treats e_ident[EI_ABIVERSION] as a feature level requested by the shared object (executable or dynamic library) and refuses to load it if an unknown feature is requested, i.e. e_ident[EI_ABIVERSION] is greater than the largest known feature. ^[9]
0x09		7		e_ident[EI_PAD]	Reserved padding bytes. Currently unused. Should be filled with zeros and ignored when read.
0x10		2		e_type	Identifies object file type. Value Type Meaning

0x12		2		e_machine	Specifies target <u>instruction set architecture</u> . Some examples are: Value ISA
0x14		4		e_version	Set to 1 for the original version of ELF.
0x18		4	8	e_entry	This is the memory address of the <u>entry point</u> from where the process starts executing. This field is either 32 or 64 bits long, depending on the format defined earlier (byte 0x04). If the file doesn't have an associated entry point, then this holds zero.
0x1C	0x20	4	8	e_phoff	Points to the start of the program header table. It usually follows the file header immediately following this one, making the offset 0x34 or 0x40 for 32- and 64-bit ELF executables, respectively.
0x20	0x28	4	8	e_shoff	Points to the start of the section header table.
0x24	0x30	4		e_flags	Interpretation of this field depends on the target architecture.
0x28	0x34	2		e_ehsize	Contains the size of this header, normally 64 Bytes for 64-bit and 52 Bytes for 32-bit format.
0x2A	0x36	2		e_phentsize	Contains the size of a program header table entry. As explained below, this will typically be 0x20 (32-bit) or 0x38 (64-bit).
0x2C	0x38	2		e_phnum	Contains the number of entries in the program header table.
0x2E	0x3A	2		e_shentsize	Contains the size of a section header table entry. As explained below, this will typically be 0x28 (32-bit) or 0x40 (64-bit).
0x30	0x3C	2		e_shnum	Contains the number of entries in the section header table.
0x32	0x3E	2		e_shstrndx	Contains index of the section header table entry that contains the section names.
0x34	0x40				End of ELF Header (size).

Example hexdump

00000000	7f 45 4c 46 02 01 01 00	00 00 00 00 00 00 00 00	.ELF.....
00000010	02 00 3e 00 01 00 00 00	c5 48 40 00 00 00 00 00	..>.....H@....

Program header

The program header table tells the system how to create a process image. It is found at file offset `e_phoff`, and consists of `e_phnum` entries, each with size `e_phentsize`. The layout is slightly different in 32-bit ELF vs 64-bit ELF, because the `p_flags` are in a different structure location for alignment reasons. Each entry is structured as:

Program header^[11]

Offset		Size (bytes)		Field	Purpose
32-bit	64-bit	32-bit	64-bit		
0x00		4		p_type	Identifies the type of the segment. Value Name Meaning
	0x04		4	p_flags	Segment-dependent flags (position for 64-bit structure). Value Name Meaning
0x04	0x08	4	8	p_offset	Offset of the segment in the file image.
0x08	0x10	4	8	p_vaddr	Virtual address of the segment in memory.
0x0C	0x18	4	8	p_paddr	On systems where physical address is relevant, reserved for segment's physical address.
0x10	0x20	4	8	p_filesz	Size in bytes of the segment in the file image. May be 0.
0x14	0x28	4	8	p_memsz	Size in bytes of the segment in memory. May be 0.
0x18		4		p_flags	Segment-dependent flags (position for 32-bit structure). See above p_flags field for flag definitions.
0x1C	0x30	4	8	p_align	0 and 1 specify no alignment. Otherwise should be a positive, integral power of 2, with p_vaddr equating p_offset modulus p_align.
0x20	0x38				End of Program Header (size).

Section header

Offset		Size (bytes)		Field	Purpose			
32-bit	64-bit	32-bit	64-bit					
0x00		4		sh_name	An offset to a string in the .shstrtab section that represents the name of this section.			
0x04		4		sh_type	Identifies the type of this header. <table><tr><td>Value</td><td>Name</td><td>Meaning</td></tr></table>	Value	Name	Meaning
Value	Name	Meaning						
0x08		4	8	sh_flags	Identifies the attributes of the section. <table><tr><td>Value</td><td>Name</td><td>Meaning</td></tr></table>	Value	Name	Meaning
Value	Name	Meaning						
0x0C	0x10	4	8	sh_addr	Virtual address of the section in memory, for sections that are loaded.			
0x10	0x18	4	8	sh_offset	Offset of the section in the file image.			
0x14	0x20	4	8	sh_size	Size in bytes of the section. May be 0.			
0x18	0x28	4		sh_link	Contains the section index of an associated section. This field is used for several purposes, depending on the type of section.			
0x1C	0x2C	4		sh_info	Contains extra information about the section. This field is used for several purposes, depending on the type of section.			
0x20	0x30	4	8	sh_addralign	Contains the required alignment of the section. This field must be a power of two.			
0x24	0x38	4	8	sh_entsize	Contains the size, in bytes, of each entry, for sections that contain fixed-size entries. Otherwise, this field contains zero.			
0x28	0x40				End of Section Header (size).			

Tools

- readelf is a Unix binary utility that displays information about one or more ELF files. A free software implementation is provided by GNU Binutils.
- elfutils provides alternative tools to GNU Binutils purely for Linux.^[12]
- elfdump is a command for viewing ELF information in an ELF file, available under Solaris and FreeBSD.
- objdump provides a wide range of information about ELF files and other object formats. objdump uses the Binary File Descriptor library as a back-end to structure the ELF data.
- The Unix file utility can display some information about ELF files, including the instruction set architecture for which the code in a relocatable, executable, or shared object file is intended, or on which an ELF core dump was produced.

Applications

Unix-like systems

The ELF format has replaced older executable formats in various environments. It has replaced a.out and COFF formats in Unix-like operating systems:

- Linux
- Solaris / Illumos
- IRIX
- FreeBSD^[13]
- NetBSD
- OpenBSD
- Redox
- DragonFly BSD
- Syllable
- HP-UX (except for 32-bit PA-RISC programs which continue to use SOM)
- QNX Neutrino
- MINIX^[14]

Non-Unix adoption

ELF has also seen some adoption in non-Unix operating systems, such as:

- OpenVMS, in its Itanium and amd64 versions^[15]
- BeOS Revision 4 and later for x86 based computers (where it replaced the Portable Executable format; the PowerPC version stayed with Preferred Executable Format)
- Haiku, an open source reimplementation of BeOS
- RISC OS^[16]
- Stratus VOS, in PA-RISC and x86 versions
- SkyOS
- Fuchsia OS
- Z/TPF
- HPE NonStop OS^[17]
- Deos

Microsoft Windows also uses the ELF format, but only for its Windows Subsystem for Linux compatibility system.^[18]

Game consoles

Some game consoles also use ELF:

- PlayStation Portable,^[19] PlayStation Vita, PlayStation, PlayStation 2, PlayStation 3, PlayStation 4, PlayStation 5
- GP2X
- Dreamcast
- GameCube
- Nintendo 64
- Wii
- Wii U

PowerPC

Other (operating) systems running on PowerPC that use ELF:

- AmigaOS 4, the ELF executable has replaced the prior Extended Hunk Format (EHF) which was used on Amigas equipped with PPC processor expansion cards.
- MorphOS
- AROS
- Café OS (The operating system run by the Wii U)

Mobile phones

Some operating systems for mobile phones and mobile devices use ELF:

- Symbian OS v9 uses E32Image^[20] format that is based on the ELF file format;
- Sony Ericsson, for example, the W800i, W610, W300, etc.
- Siemens, the SGOLD and SGOLD2 platforms: from Siemens C65 to S75 and BenQ-Siemens E71/EL71;
- Motorola, for example, the E398, SLVR L7, v360, v3i (and all phone LTE2 which has the patch applied).
- Bada, for example, the Samsung Wave S8500.
- Nokia phones or tablets running the Maemo or the Meego OS, for example, the Nokia N900.
- Android uses ELF .so (shared object^[21]) libraries for the Java Native Interface. With Android Runtime (ART), the default since Android 5.0 "Lollipop", all applications are compiled into native ELF binaries on installation.^[22] It's also possible to use native Linux software from package managers like Termux, or compile them from sources via Clang or GCC, that are available in repositories.

Some phones can run ELF files through the use of a patch that adds assembly code to the main firmware, which is a feature known as *ELFPack* in the underground modding culture. The ELF file format is also used with the Atmel AVR (8-bit), AVR32^[23] and with Texas Instruments MSP430 microcontroller architectures. Some implementations of Open Firmware can also load ELF files, most notably Apple's implementation used in almost all PowerPC machines the company produced.

Blockchain platforms

- Solana uses ELF format for its on-chain programs (smart contracts). The platform processes ELF files compiled to BPF (Berkeley Packet Filter) byte-code, which are then deployed as shared

objects and executed in Solana's runtime environment. The BPF loader validates and processes these ELF files during program deployment.^[24]

Specifications

Architecture	Specification
Generic	▪ <i>System V Application Binary Interface</i> (http://www.sco.com/developers/devspecs/gabi41.pdf) Edition 4.1 (1997-03-18) ▪ <i>System V ABI Update</i> (http://www.sco.com/developers/gabi/latest/contents.html) (October 2009)
<u>AMD64</u>	▪ <i>System V ABI, AMD64 Supplement</i> (http://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf)
<u>Arm</u>	▪ <i>ELF for the ARM Architecture</i> (https://github.com/ARM-software/abi-aa/releases/download/2022Q1/aaelf32.pdf)
<u>IA-32</u>	▪ <i>System V ABI, Intel386 Architecture Processor Supplement</i> (http://www.sco.com/developers/devspecs/abi386-4.pdf)
<u>IA-64</u>	▪ <i>Itanium Software Conventions and Runtime Guide</i> (http://refspecs.linux-foundation.org/IA64conventions.pdf) (September 2000)
<u>M32R</u>	▪ <i>M32R ELF ABI Supplement</i> (http://www.linux-m32r.org/cmn/m32r/M32R-elf-abi.pdf) Archived (https://web.archive.org/web/20220512145623/http://www.linux-m32r.org/cmn/m32r/M32R-elf-abi.pdf) 2022-05-12 at the Wayback Machine Version 1.2 (2004-08-26)
<u>MIPS</u>	▪ <i>System V ABI, MIPS RISC Processor Supplement</i> (http://www.sco.com/developers/devspecs/mipsabi.pdf) ▪ <i>MIPS EABI documentation</i> (http://sources.redhat.com/ml/binutils/2003-06/msg00436.html) Archived (https://web.archive.org/web/20120401235051/http://sources.redhat.com/ml/binutils/2003-06/msg00436.html) 2012-04-01 at the Wayback Machine (2003-06-11)
<u>Motorola 6800</u>	▪ <i>Motorola 8- and 16- bit Embedded ABI</i> (http://uclibc.org/docs/psABI-m8-16.pdf)
<u>PA-RISC</u>	▪ <i>ELF Supplement for PA-RISC</i> (https://web.archive.org/web/20110317045038/http://refspecs.freestandards.org/elf/elf-pa.pdf) Version 1.43 (October 6, 1997)
<u>PowerPC</u>	▪ <i>System V ABI, PPC Supplement</i> (https://web.archive.org/web/20070630123210/http://refspecs.freestandards.org/elf/elfspec_ppc.pdf) ▪ <i>PowerPC Embedded Application Binary Interface</i> (https://web.archive.org/web/20110723003758/http://sources-redhat.mirrors.airband.net/binutils/ppc-docs/ppc-eabi-1995-01.pdf) 32-Bit Implementation (1995-10-01) ▪ <i>64-bit PowerPC ELF Application Binary Interface Supplement</i> (http://refspecs.linuxfoundation.org/ELF/ppc64/PPC-elf64abi-1.9.html) Version 1.9 (2004)
<u>RISC-V</u>	▪ <i>RISC-V ELF Specification</i> (https://github.com/riscv-non-isa/riscv-elf-psabi-doc/blob/master/riscv-elf.adoc)
<u>SPARC</u>	▪ <i>System V ABI, SPARC Supplement</i> (https://web.archive.org/web/20080517110249/http://www.sparc.org/standards/psABI3rd.pdf)
<u>S/390</u>	▪ <i>S/390 32bit ELF ABI Supplement</i> (http://refspecs.linuxbase.org/ELF/zSeries/lzsabi0_s390.html)

<u>zSeries</u>	▪ <i>zSeries 64bit ELF ABI Supplement</i> (http://refspecs.linuxbase.org/ELF/zSeries/lzsabi0_zSeries.html)
<u>Symbian OS 9</u>	▪ <i>E32Image file format on Symbian OS 9</i> (https://web.archive.org/web/20080518002831/http://wiki.forum.nokia.com/index.php/E32Image_file_format_on_Symbian_OS_9)

The Linux Standard Base (LSB) supplements some of the above specifications for architectures in which it is specified.^[25] For example, that is the case for the System V ABI, AMD64 Supplement.^{[26][27]}

86open

86open was a project to form consensus on a common binary file format for Unix and Unix-like operating systems on the common PC compatible x86 architecture, to encourage software developers to port to the architecture.^[28] The initial idea was to standardize on a small subset of Spec 1170, a predecessor of the Single UNIX Specification, and the GNU C Library (glibc) to enable unmodified binaries to run on the x86 Unix-like operating systems. The project was originally designated "Spec 150".

The format eventually chosen was ELF, specifically the Linux implementation of ELF, after it had turned out to be a *de facto* standard supported by all involved vendors and operating systems.

The group began email discussions in 1997 and first met together at the Santa Cruz Operation offices on August 22, 1997.

The steering committee was Marc Ewing, Dion Johnson, Evan Leibovitch, Bruce Perens, Andrew Roach, Bryan Wayne Sparks and Linus Torvalds. Other people on the project were Keith Bostic, Chuck Cranor, Michael Davidson, Chris G. Demetriou, Ulrich Drepper, Don Dugger, Steve Ginzburg, Jon "maddog" Hall, Ron Holt, Jordan Hubbard, Dave Jensen, Kean Johnston, Andrew Josey, Robert Lipe, Bela Lubkin, Tim Marsland, Greg Page, Ronald Joe Record, Tim Ruckle, Joel Silverstein, Chia-pi Tien, and Erik Troan. Operating systems and companies represented were BeOS, BSDI, FreeBSD, Intel, Linux, NetBSD, SCO and SunSoft.

The project progressed and in mid-1998, SCO began developing lxrund, an open-source compatibility layer able to run Linux binaries on OpenServer, UnixWare, and Solaris. SCO announced official support of lxrund at LinuxWorld in March 1999. Sun Microsystems began officially supporting lxrund for Solaris in early 1999,^[29] and later moved to integrated support of the Linux binary format via Solaris Containers for Linux Applications.

With the BSDs having long supported Linux binaries (through a compatibility layer) and the main x86 Unix vendors having added support for the format, the project decided that Linux ELF was the format chosen by the industry and "declare[d] itself dissolved" on July 25, 1999.^[30]

FatELF: universal binaries for Linux

FatELF is an ELF binary-format extension that adds fat binary capabilities.^[31] It is aimed for Linux and other Unix-like operating systems. Additionally to the CPU architecture abstraction (byte order, word size, CPU instruction set etc.), there is the potential advantage of software-platform abstraction e.g., binaries which

support multiple kernel ABI versions. As of 2021, FatELF has not been integrated into the mainline Linux kernel.^{[32][33][34]}

See also

- [Application binary interface](#)
- [Comparison of executable file formats](#)
- [DWARF](#) – a format for debugging data
- [Intel Binary Compatibility Standard](#)
- [Portable Executable](#) – format used by Windows
- [vDSO](#) – virtual DSO
- [Position-independent code](#)



References

1. Tool Interface Standard (TIS) *Executable and Linking Format (ELF) Specification* (<https://refspecs.linuxbase.org/elf/elf.pdf>) Version 1.2 (May 1995)
2. ELF Object File Format: Appendix C: Revision History (<https://gabi.xinuos.com/elf/c-history.html>)
3. Tool Interface Standard (TIS) *Portable Formats Specification* (<https://refspecs.linuxfoundation.org/elf/TIS1.1.pdf>) Version 1.1 (October 1993)
4. *System V Application Binary Interface* (<https://www.sco.com/developers/devspecs/gabi41.pdf>) Edition 4.1 (1997-03-18)
5. "ELF Header" (<https://www.sco.com/developers/gabi/2000-07-17/ch4.eheader.html>). Sco.com. July 2000. Retrieved 2014-02-07.
6. "LXR linux/include/linux/elf.h" (<http://lxr.linux.no/linux+v2.6.11/include/linux/elf.h#L380>). *linux.no*. Retrieved 27 April 2015.
7. "glibc 2.12 announce" (<https://sourceware.org/ml/libc-alpha/2010-05/msg00000.html>).
8. "sourceware.org Git - glibc.git/blob - libc-abis" (<https://sourceware.org/git/?p=glibc.git;a=blob;f=libc-abis;h=e702f6ae245c1528f5a608df8cfae4f037809de2;hb=HEAD>).
9. "sourceware.org Git - glibc.git/blob - sysdeps/gnu/ldsodefs.h" (<https://web.archive.org/web/20210307223850/https://sourceware.org/git/?p=glibc.git%3Ba%3Dblob%3Bf%3Dsysdeps%2Fgnu%2Fldsodefs.h%3Bh%3D253b4d934c5ca457ab5580a2a6398205e4f961ba%3Bhb%3DHEAD#I32>). Archived from the original (<https://sourceware.org/git/?p=glibc.git%3Ba%3Dblob%3Bf%3Dsysdeps%2Fgnu%2Fldsodefs.h%3Bh%3D253b4d934c5ca457ab5580a2a6398205e4f961ba%3Bhb%3DHEAD#I32>) on 2021-03-07. Retrieved 2019-10-28.
10. "Available lexers — Pygments" (<https://pygments.org/docs/lexers/#lexers-for-hexadecimal-dumps>). *pygments.org*. Retrieved 2024-12-19.
11. "Program Header" (<http://www.sco.com/developers/gabi/2000-07-17/ch5.pheader.html>). Sco.com. July 2000. Retrieved 2017-04-05.
12. "elfutils" (<https://sourceware.org/elfutils/>). *sourceware.org*. Retrieved 30 April 2017.
13. "Binary Formats" (<https://web.archive.org/web/20190331192807/https://docs.freebsd.org/doc/4.9-RELEASE/usr/share/doc/handbook/binary-formats.html>). Archived from the original (<https://docs.freebsd.org/doc/4.9-RELEASE/usr/share/doc/handbook/binary-formats.html>) on 2019-03-31. Retrieved 2019-03-31.

14. "MinixReleases – Minix Wiki" (<https://web.archive.org/web/20130330150621/http://wiki.minix3.org/en/MinixReleases>). Wiki.minix3.org. Archived from the original (<http://wiki.minix3.org/en/MinixReleases>) on 2013-03-30. Retrieved 2014-01-19.
15. "Archived copy" (https://web.archive.org/web/20200915215111/https://vmssoftware.com/pdfs/State_of_Port_20160906.pdf) (PDF). Archived from the original (https://vmssoftware.com/pdfs/State_of_Port_20160906.pdf) (PDF) on 2020-09-15. Retrieved 2016-10-19.
16. "GCCSDK – RISC OS" (<https://web.archive.org/web/20140219211405/http://www.riscos.info/index.php/GCCSDK>). Riscos.info. 2012-04-22. Archived from the original (<http://www.riscos.info/index.php/GCCSDK>) on 2014-02-19. Retrieved 2014-01-19.
17. "Guardian Programmer's Guide" (https://web.archive.org/web/20180530042640/http://h20628.www2.hp.com/km-ext/kmcsdirect/emr_na-c02543407-12.pdf) (PDF). Hewlett Packard Enterprise. Archived from the original (http://h20628.www2.hp.com/km-ext/kmcsdirect/emr_na-c02543407-12.pdf) (PDF) on 2018-05-30. Retrieved 2018-05-30. p. 44 archived from the original (http://h20628.www2.hp.com/km-ext/kmcsdirect/emr_na-c02543407-12.pdf) Archived (https://web.archive.org/web/20180530042640/http://h20628.www2.hp.com/km-ext/kmcsdirect/emr_na-c02543407-12.pdf) 2018-05-30 at the Wayback Machine on 2018-5-30
18. Foley, Mary Jo. "Under the hood of Microsoft's Windows Subsystem for Linux" (<https://www.zdnet.com/article/under-the-hood-of-microsofts-windows-subsystem-for-linux/>). ZDNet. Retrieved 2016-08-19.
19. PlayStation Portable use encrypted & relocated ELF : PSP
20. *Symbian OS executable file format* (<https://web.archive.org/web/20091213034509/http://wiki.forum.nokia.com/index.php/E32Image>)
21. Rosen, Kenneth; Host, Douglas; Klee, Rachel; Rosinski, Richard (2007). *UNIX: The Complete Reference* (<https://books.google.com/books?id=2Et--84HlkWC>) (2 ed.). McGraw Hill Professional. p. 707. ISBN 9780071706988. Retrieved 2017-06-08. "Dynamically linked libraries are also called shared objects (.so)."
22. Thomas, Romain. "Android formats" (https://lief-project.github.io/doc/latest/tutorials/10_android_formats.html). Quarks Lab. Retrieved 17 Jan 2023.
23. "Chapter 4: Object Files" (http://www.sco.com/developers/gabi/2009-10-26/ch4.eheader.html#e_machine), *System V Application Binary Interface*, 2009-10-26, e_machine
24. "Solana Programs" (<https://solana.com/docs/core/programs>). Solana Foundation. 2024-11-11. Retrieved 2024-11-11. "Solana leverages the LLVM compiler infrastructure to compile programs into Executable and Linkable Format (ELF) files."
25. "LSB Referenced Specifications" (<http://refspecs.linuxfoundation.org/lbs.shtml>). *linuxfoundation.org*. Retrieved 27 April 2015.
26. "Executable and Linking Format (ELF)" (http://refspecs.linuxfoundation.org/LSB_4.1.0/LSB-Core-AMD64/LSB-Core-AMD64/elf-amd64.html). *linuxfoundation.org*. Retrieved 27 April 2015.
27. "Introduction" (http://refspecs.linuxfoundation.org/LSB_4.1.0/LSB-Core-AMD64/LSB-Core-AMD64/elfintro.html). *linuxfoundation.org*. Retrieved 27 April 2015.
28. Leibovitch, Evan (1997-12-23). "86Open Frequently-Asked Questions" (<https://web.archive.org/web/20070311032337/http://www.telly.org/86open-faq>). Archived from the original (<http://www.telly.org/86open-faq>) on 2007-03-11. Retrieved 2007-06-06.
29. Record, Ronald (1998-05-21). "Bulletin on status of 86open at SCO" (https://web.archive.org/web/20081208013909/http://www.mavetju.org/mail/view_message.php?list=freebsd-emulation&id=361608). Archived from the original (http://www.mavetju.org/mail/view_message.php?list=freebsd-emulation&id=361608) on 2008-12-08. Retrieved 2008-05-06.
30. Leibovitch, Evan (1999-07-25). "The86open Project – Final Update" (<https://web.archive.org/web/20070227214032/http://www.telly.org/86open/>). Archived from the original (<https://www.telly.org/86open/>) on 2007-02-27. Retrieved 2007-05-06.

31. Gordon, Ryan. "fatelf-specification v1" (<http://hg.icculus.org/icculus/fatelf/raw-file/tip/docs/fatelf-specification.txt>). icculus.org. Retrieved 2010-07-25.
32. Gordon, Ryan. "FatELF: Turns out I liked the uncertainty better" (<https://icculus.org/cgi-bin/finger/finger.pl?user=icculus&date=2009-11-03&time=19-08-04>). icculus.org. Retrieved 2010-07-13.
33. Holwerda, Thom (2009-11-03). "Ryan Gordon Halts FatELF Project" (<https://www.osnews.com/story/22446/ryan-gordon-halts-fatelf-project>). osnews.com. Retrieved 2010-07-05.
34. Brockmeier, Joe (June 23, 2010). "SELF: Anatomy of an (alleged) failure" (<https://lwn.net/Articles/392862/>). Linux Weekly News. Retrieved 2011-02-06.

Further reading

- Evolution of the ELF object file format (<https://maskray.me/blog/2024-05-26-evolution-of-elf-object-file-format>), by Fangrui Song.
- Levine, John R. (2000) [October 1999]. *Linkers and Loaders* (<https://www.iecc.com/linker/>). The Morgan Kaufmann Series in Software Engineering and Programming (1 ed.). San Francisco, USA: Morgan Kaufmann. ISBN 1-55860-496-0. OCLC 42413382 (<https://search.worldcat.org/oclc/42413382>). Retrieved 2020-01-12. Code:^[1][1] (<ftp://ftp.iecc.com/pub/linker/>) Errata: [2] (<https://linker.iecc.com/>)
- Ulrich Drepper, How To Write Shared Libraries, Version 4.1.2 (2011). Published on the author's web page, <https://www.akkadia.org/drepper>.
- *An unsung hero: The hardworking ELF* (<https://web.archive.org/web/20070224140341/http://www-128.ibm.com/developerworks/power/library/pa-spec12/>) by Peter Seebach, December 20, 2005, archived from the original on February 24, 2007
- LibElf and GElf – A Library to Manipulate ELF Files (<https://web.archive.org/web/20040225174057/http://developers.sun.com/solaris/articles/elf.html>) at the Wayback Machine (archived February 25, 2004)
- *The ELF Object File Format: Introduction* (<https://www.linuxjournal.com/article/1059>), *The ELF Object File Format by Dissection* (<https://www.linuxjournal.com/article/1060>) by Eric Youngdale (1995-05-01)
- *A Whirlwind Tutorial on Creating Really Teensy ELF Executables for Linux* (<https://www.muppetlabs.com/~breadbox/software/tiny/teensy.html>) by Brian Raiter
- ELF relocation into non-relocatable objects (<https://www.phrack.org/issues.html?issue=61&id=8#article>) by Julien Vanegue (2003-08-13)
- Embedded ELF debugging without ptrace (<https://www.phrack.org/issues.html?issue=63&id=9#article>) by the ELFsh team (2005-08-01)
- *Study of ELF loading and relocs* (https://netwinder.osuosl.org/users/p/pathb/public_html/elf_relocs.html) by Pat Beirne (1999-08-03)

External links

- ELF Object File Format Specification (<https://gabi.xinuos.com>)
- ELF Object File Format (<https://github.com/xinuos/gabi>) on GitHub
- System V Application Binary Interface and Interface Definition (<https://www.sco.com/developers/ddefs/defs/>)
- FreeBSD Handbook: Binary formats (https://web.archive.org/web/20130403001804/http://www.freebsd.org/doc/en_US.ISO8859-1/books/handbook/binary-formats.html) (archived version)
- FreeBSD elf(5) manual page (<https://www.freebsd.org/cgi/man.cgi?query=elf&sektion=5>)

- [NetBSD ELF FAQ \(https://www.netbsd.org/Documentation/elf.html\)](https://www.netbsd.org/Documentation/elf.html)
 - [Linux elf\(5\) manual page \(https://man7.org/linux/man-pages/man5/elf.5.html\)](https://man7.org/linux/man-pages/man5/elf.5.html)
 - [Oracle Solaris Linker and Libraries Guide \(https://docs.oracle.com/en/operating-systems/solaris/oracle-solaris/11.4/linkers-libraries/\)](https://docs.oracle.com/en/operating-systems/solaris/oracle-solaris/11.4/linkers-libraries/)
 - [The ERESI project: reverse engineering on ELF-based operating systems \(http://www.eresi-project.org/\)](http://www.eresi-project.org/) Archived (<https://web.archive.org/web/20210314190846/http://www.eresi-project.org/>) 2021-03-14 at the [Wayback Machine](https://web.archive.org/web/20210314190846/http://www.eresi-project.org/)
 - [Linux Today article on 86open \(https://www.linuxtoday.com/developer/1999072600605PS\)](https://www.linuxtoday.com/developer/1999072600605PS) Archived (<https://web.archive.org/web/20210308053629/http://www.linuxtoday.com/developer/1999072600605PS>) 2021-03-08 at the [Wayback Machine](https://web.archive.org/web/20210308053629/http://www.linuxtoday.com/developer/1999072600605PS) July 26, 1999
 - [Announcement of 86open on Debian Announce mailing list \(https://lists.debian.org/debian-announce/1997/msg00028.html\)](https://lists.debian.org/debian-announce/1997/msg00028.html) October 10, 1997, Bruce Perens
 - [Declaration of Ulrich Drepper \(PDF\) \(https://web.archive.org/web/20241130193343/https://www.groklaw.net/pdf/IBM-835-Exhibit_184.pdf\)](https://web.archive.org/web/20241130193343/https://www.groklaw.net/pdf/IBM-835-Exhibit_184.pdf) in [The SCO Group vs IBM](https://web.archive.org/web/20241130193343/https://www.groklaw.net/pdf/IBM-835-Exhibit_184.pdf), September 19, 2006
 - [86open and ELF discussion \(https://web.archive.org/web/20190201013659/http://www.groklaw.net/articlebasic.php?story=20060813114048520\)](https://web.archive.org/web/20190201013659/http://www.groklaw.net/articlebasic.php?story=20060813114048520) on [Groklaw](https://web.archive.org/web/20190201013659/http://www.groklaw.net/articlebasic.php?story=20060813114048520), August 13, 2006
1. <https://archive.today/20200114225034/https://linker.iecc.com/code.html>

Retrieved from "https://en.wikipedia.org/w/index.php?title=Executable_and_Linkable_Format&oldid=1353635085"